Phase 12: Controlled IDOR Vulnerability Demonstration & Remediation

Course Code: 20CYS401 | **Project**: Online Book Review and Recommendation Platform

---

12.1 Background & Threat Context

- **Vulnerability**: Insecure Direct Object Reference (IDOR) / Broken Object Level Authorization (BOLA)

- **CWE**: CWE-639 (Authorization Bypass Through User-Controlled Key)

- **OWASP Top 10**: A01:2021 — Broken Access Control

- **Threat Model Mapping**: THR-02, VUL-01, ATK-01

In an online book review platform, every user must only be permitted to edit or delete reviews they personally authored. If an endpoint accepts a `review_id` and performs mutations without verifying that the requesting user matches the author record, an attacker can tamper with community trust by altering or erasing arbitrary reviews across the entire platform.

12.2 The Vulnerability (Before Fix)

In the demonstration branch `demo/idor-before`, the ownership verification step was deliberately omitted from `app/services/review_service.py`:

```
# VULNERABLE IMPLEMENTATION (on branch demo/idor-before):
def update_review(review_id, user_id, rating, content):
review = Review.query.get(review_id)
if not review:
return False, "Review not found."

# CRITICAL FLAW: Missing ownership check!
# Does not verify if review.user_id == user_id

review.rating = rating
review.content = sanitize_text(content)
db.session.commit()
return True, "Review updated successfully."
```

Exploit Proof Test

On branch `demo/idor-before`, `tests/test_idor_vulnerability_proof.py` executes:

1. User Alice authors a legitimate review with rating=5.

2. User Bob (attacker) issues an edit request with rating=1 and malicious text targeting Alice's review ID.

3. Because the vulnerable service omits ownership validation, Bob's request modifies Alice's review in the database.

4. **Result**: `tests/test_idor_vulnerability_proof.py` **PASSED** (proving the vulnerability exists and is exploitable).

- **Evidence**: `evidence/phase-12/idor_vulnerable_proof.txt`

12.3 The Remediation (After Fix — On `main`)

On the `main` branch, strict server-side object ownership verification is enforced inside `ReviewService`:

```
# SECURE IMPLEMENTATION (on branch main):
def update_review(review_id, user_id, rating, content):
review = Review.query.get(review_id)
if not review:
```

```
return False, "Review not found."

# SECURE FIX: CTRL-02 Object-level ownership validation
if not review.is_owned_by(user_id):
log_audit_event(
action="UNAUTHORIZED_REVIEW_EDIT_ATTEMPT",
resource_type="review",
resource_id=str(review_id),
status="DENIED",
details=f"User {user_id} attempted to edit review owned by user {review.user_id}"
)
return False, "Access denied: You can only edit your own reviews."

review.rating = rating
review.content = sanitize_text(content)
db.session.commit()
log_audit_event("REVIEW_UPDATED", resource_type="review", resource_id=str(review_id), status="SUCCESS")
return True, "Review updated successfully."
```

Verification on `main`

On the `main` branch, running `tests/test_authorization.py`:

- `test_user_b_cannot_edit_user_a_review` **PASSED** (Bob's tamper request rejected, returns False/Access Denied, database unchanged).

- `test_user_b_cannot_delete_user_a_review` **PASSED** (Bob's deletion request rejected, Alice's review remains intact).

- **Evidence**: `evidence/phase-12/idor_fix_verification.txt`

12.4 Comparison Summary

| Parameter | Vulnerable State (`demo/idor-before`) | Secure State (`main`) |

|---|---|---|

| Service Logic | Directly commits updates if `review_id` exists | Validates `review.is_owned_by(user_id)` before mutation |

| HTTP Result | Attacker Bob succeeds in editing Alice's review | Attacker Bob receives Access Denied |

| Database State | Review overwritten with attacker content | Review preserved with original author's content |

| Audit Trail | None or misleading event logged | `UNAUTHORIZED_REVIEW_EDIT_ATTEMPT` recorded with status `DENIED` |

| Pytest Evidence | `evidence/phase-12/idor_vulnerable_proof.txt` (PASSED) | `evidence/phase-12/idor_fix_verification.txt` (PASSED) |